



UNIVERSITY
OF COLOGNE

Q&A SESSION

Advanced Analytics and Applications

Advertisement

Ready to code for a better future?

Then join the @BCG Platinion #Hackathon2026 and work alongside tech talent from around the world to develop innovative solutions to global hunger. From food distribution to digital nutrition monitoring – your ideas could save lives.

Work side by side with AI tech consultants and architects from BCG Platinion, gain first-hand insights into IT strategy consulting, and find out more about your career opportunities.

 16–17 October

 Frankfurt

 Apply now:

https://www.bcgplatinion.com/hackathon?utm_source=university&utm_medium=referral&utm_campaign=hackathon-2026&utm_content=unik%C3%B6ln&local=de

The hackathon will take place at hubs near our global offices. Applicants currently residing in Germany, Austria or Switzerland will be invited to the hub in Frankfurt. There is no participation fee. BCG Platinion will cover all approved event-related travel and accommodation costs.



Institut / Lehrstuhl / Dezernat



22.07.26

Agenda

1

Submitted Questions

2

Final Remarks

Kernel Functions

Question: Is there a general (simple) definition of kernel machines or does the term “kernel machines” simply refer to the projection of data?

Answer: Kernel machines are machine learning *algorithms/frameworks* that use pairwise similarity evaluation via a kernel function rather than computing these in projected space. So it is explicitly **not** simply the projection of data, but the trick to not need do this explicitly. A kernel machine is the kernelized variant of a linear ML model: Linear SVM → Kernel SVM

Soft Clustering

Question: What is the difference between fuzzy c-means and the Gaussian mixture model?

Answer:

- **Fuzzy c-means**^{*} is an extension of k-means that uses deterministic degrees of membership. Each point holds a vector of membership degree to each cluster. That degree is based on distance to cluster centers. As such the assumed cluster boundaries are spherical.
- A **Gaussian Mixture Model (GMM)** is a probabilistic, iterative (EM algorithm) clustering method for distribution fitting that makes use of the fact that we can estimate the likelihood that points could've been generated from the mixture of the distributions we assume along the input dimensions. GMM can handle non-spherical cluster boundaries with a modified covariance matrix.

^{*}We didn't talk about this in AAA, this is from AA.

Markov Decision Problems

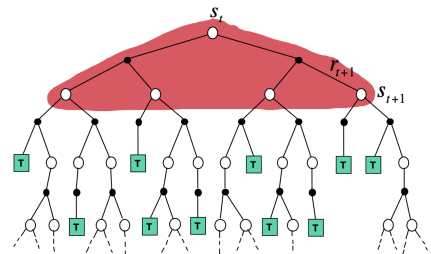
Question: Could you please give us a structured overview of Dynamic Programming, Monte Carlo, and Temporal Difference methods? Or can you provide us with more calculation exercises so we can better understand the formulas/concepts of these methods? I'm still struggling to understand them.

Answer:

1 Dynamic Programming (DP)

- Require a complete model of the environment
- Well-developed mathematically
- Guaranteed optimal policy
- Not suitable for large problems
- No sampling, no bootstrapping

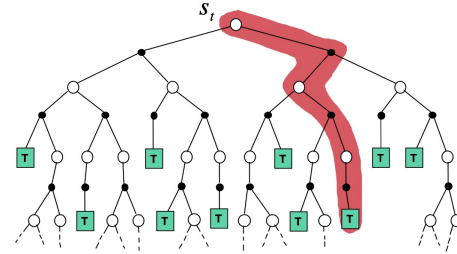
$$V(S_t) \leftarrow \mathbb{E}_{\pi}[R_{t+1} + \gamma V(S_{t+1})]$$



2 Monte Carlo (MC)

- Learn directly from complete episodes of experience
- Don't require a model
- Conceptually simple
- Only suitable for episodic MDPs
- Sampling, no bootstrapping

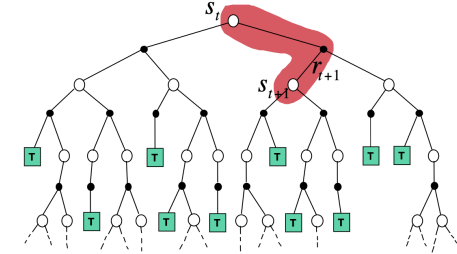
$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$



3 Temporal-difference (TD)

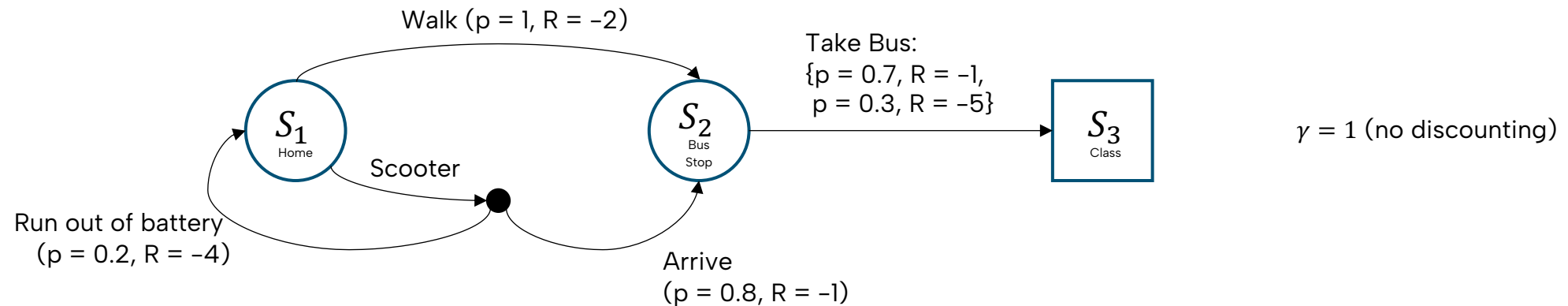
- Require no model
- Learn from incomplete episodes
- Fully incremental
- More complex to analyse
- Sampling and bootstrapping

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$



Example: Markov Decision Process

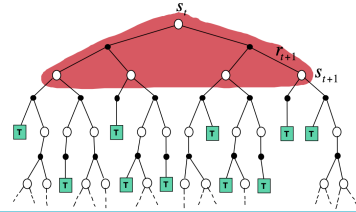
Let's look at a simple MDP which we fully know the environment dynamics of:



Example: Solving with Dynamic Programming

(Policy Iteration)

Lecture 11, Slide 39



Policy Evaluation (Prediction Task)

Let's assume a policy π that is:
 $\pi(S_1) = \text{Scooter}$ and $\pi(S_2) = \text{Bus}$

Initialize $V(S_1) = V(S_2) = 1$, $V(S_3) = 0$

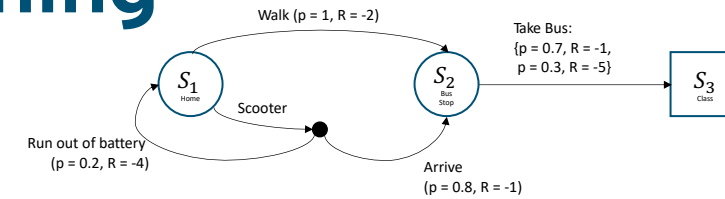
To evaluate this policy, we do a full sweep of all states with the Bellman Expectation Equation:

$$V(S_1) \leftarrow \mathbb{E}[R + \gamma V(S_2)] = 0.8(-1 + V(S_2)) + 0.2(-4 + V(S_1)) \\ = 0.8(-1 + 1) + 0.2(-4 + 1) = -0.6$$

$$V(S_2) \leftarrow \mathbb{E}[R + \gamma V(S_3)] = 0.7(-1 + 0) + 0.3(-5 + 0) = -2.2$$

Then, we iterate (repeat).

Lecture 11, Slide 42



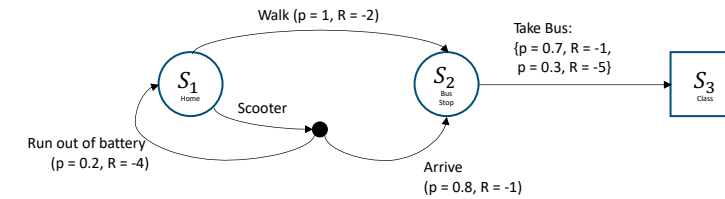
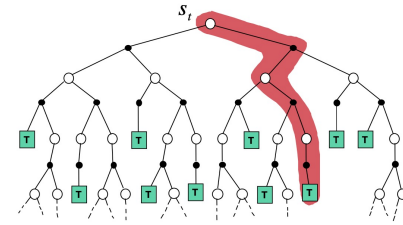
Policy Improvement (Control Task)

To improve our policy, we update it such that it is greedy w.r.t. V_π :

In this example, after the first step, the greedy update would be:

$$\pi(S_1) = \underset{a}{\operatorname{argmax}} \sum_{s', r} p(s', r | S_1, a) [r + \gamma V(s')] \\ = \underset{a}{\operatorname{argmax}} \begin{cases} 1(-2 + 1 * -2.2) & , a: \text{walk} \\ 0.8(-1 + 1 * -2.2) + 0.2(-4 + 1 * -0.6) & , a: \text{scooter} \end{cases} \\ = \underset{a}{\operatorname{argmax}} \begin{cases} -4.2 & , a: \text{walk} \\ -3.48 & , a: \text{scooter} \end{cases} = a_{\text{scooter}}$$

Example: Learning with Monte Carlo



Now, when we don't know the environment dynamics $p(s', r|s, a)$ we have to observe traces and need to resort to estimations of the value of actions. Enter **Q-Values**. We initialize $Q(s, a) = 0 \forall s, a$ and $\alpha = 0.1$

We sample one full episode where the agent picks $a_{scooter}$ at S_1 and gets stuck in a traffic jam with the bus:

$$S_1 \rightarrow a_{scooter} \rightarrow R = -1 \rightarrow S_2 \rightarrow a_{bus} \rightarrow R = -5 \rightarrow S_3$$

This gives us return G_t for the observed state-action pairs: $G_1(S_2, a_{bus}) = -5$, $G_0(S_1, a_{scooter}) = -1 + (-5) = -6$

We can use these to update Q-values for all visited state-action pairs (every-visit MC):

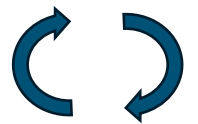
$$\begin{aligned} Q(S_1, a_{scooter}) &\leftarrow Q(S_1, a_{scooter}) + \alpha[G_0 - Q(S_1, a_{scooter})] \\ &= 0 + 0.1(-6.0 - 0) = -\mathbf{0.6} \end{aligned}$$

$$Q(S_2, a_{bus}) \leftarrow 0 + 0.1(-5.0 - 0) = -\mathbf{0.5}$$

Finally, we update the policy, at least the greedy part w.r.t. Q-values:

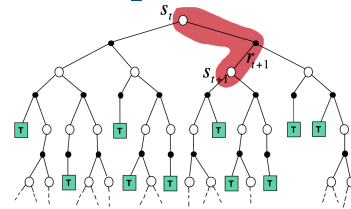
$$\pi(S_1) = \varepsilon - greedy(Q(S_1)) \rightarrow \begin{cases} random & , with p(\varepsilon) \\ argmax_a Q(S_1, a) \rightarrow a_{walk} & , with p(1 - \varepsilon) \end{cases}$$

Prediction Task

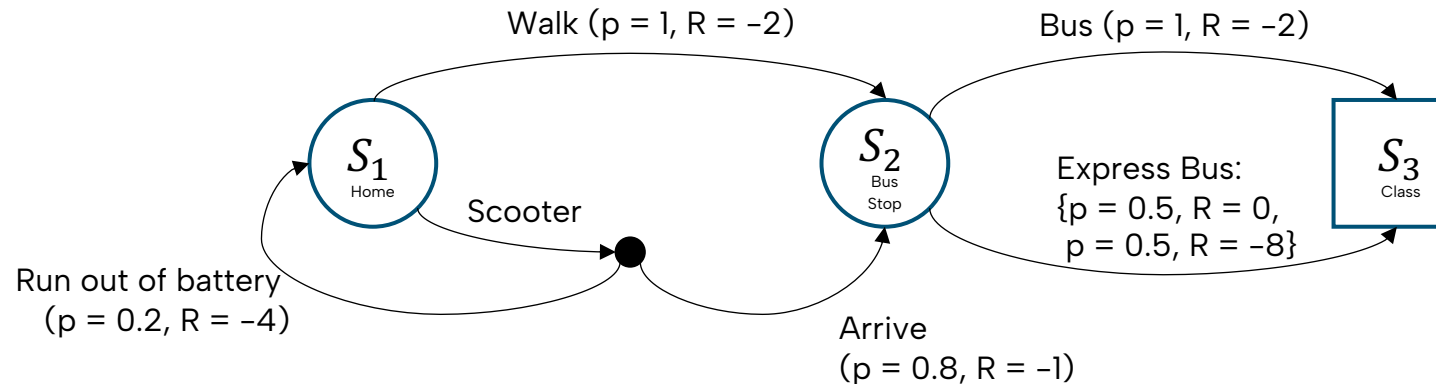


Control Task

Example: Learning with Temporal Difference



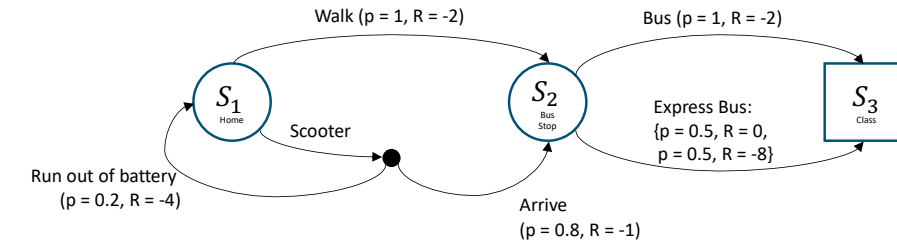
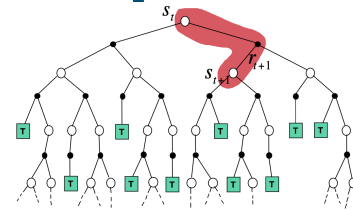
If the task at hand would not be episodic, or episodes would be very long, we should use TD methods. However, to make it interesting, let's vary the MDP a little and introduce a notoriously risky express bus:



We assume to be mid-learning with $Q(s, a) = 0 \forall s, a$ except $Q(S_2, a_{bus}) = -2$ and $Q(S_2, a_{express}) = -4$ and learning rate $\alpha = 0.5$ for illustrative purposes.

Example: Learning with Temporal Difference (SARSA, On-Policy)

Lecture 12, Slide 10



We assume to be mid-learning with $Q(s, a) = 0 \forall s, a$ except $Q(S_2, a_{bus}) = -2$ and $Q(S_2, a_{express}) = -4$ and learning rate $\alpha = 0.5$ for illustrative purposes.

We observe one single transition: $S_1 \rightarrow a_{walk} \rightarrow R = -2 \rightarrow S_2$

For SARSA, we have to check what the next action would be following the current ϵ – *greedy* policy.

Exploitation: a_{bus} (greedy w.r.t $Q(S_2, a)$)

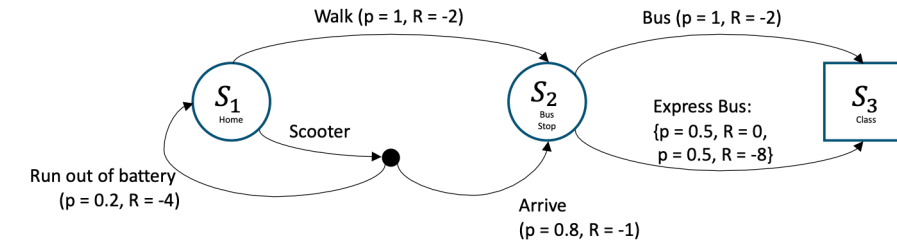
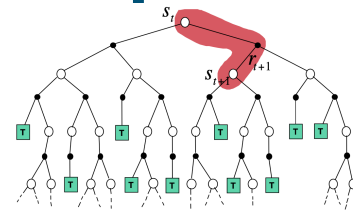
$$\begin{aligned} Q(S_1, a_{walk}) &\leftarrow Q(S_1, a_{walk}) \\ &+ \alpha [R_{t+1} + \gamma Q(S_2, a_{bus}) - Q(S_1, a_{walk})] \\ &= 0.0 + 0.5 [-2 + 1.0(-2.0) - 0.0] \\ &= 0.5 [-4.0] \\ &= \mathbf{-2.0} \end{aligned}$$

Exploration: $a_{express}$ (random)

$$\begin{aligned} Q(S_1, a_{walk}) &\leftarrow Q(S_1, a_{walk}) \\ &+ \alpha [R_{t+1} + \gamma Q(S_2, a_{express}) - Q(S_1, a_{walk})] \\ &= 0.0 + 0.5 [-2 + 1.0(-4.0) - 0.0] \\ &= 0.5 [-6.0] \\ &= \mathbf{-3.0} \end{aligned}$$

Example: Learning with Temporal Difference (Q-Learning, Off-Policy)

Lecture 12, Slide 12



We assume to be mid-learning with $Q(s, a) = 0 \forall s, a$ except $Q(S_2, a_{bus}) = -2$ and $Q(S_2, a_{express}) = -4$ and learning rate $\alpha = 0.5$ for illustrative purposes.

We observe one single transition: $S_1 \rightarrow a_{walk} \rightarrow R = -2 \rightarrow S_2$

For Q-Learning we don't care about our current policy, we simply use the update rule that considers the maximum possible Q-Value at S_2 :

$$\max_{a'} Q(S_2, a') = \max(-2, -4) = -2$$

$$Q(S_1, a_{walk}) \leftarrow Q(S_1, a_{walk}) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_2, a') - Q(S_1, a_{walk}) \right]$$

$$\begin{aligned} Q(S_1, a_{walk}) &\leftarrow 0.0 + 0.5[-2 + 1.0(-2.0) - 0.0] \\ &= 0.5[-4.0] \\ &= -2.0 \end{aligned}$$

Agenda

1

Submitted Questions

2

Final Remarks

Remarks on the exam

- Format of the exam: 90 minutes, 90 points, closed book, calculator allowed
 - Any combination of single choice questions and fill-in-the-blanks for up to 15 points in total
 - Maximum one Python question for up to 15 points in total
 - Remaining points distributed across open text and calculation tasks
- Questions have, where necessary, **keywords** that give you an instruction on how to answer:
 - **“Name/State...”**: No explanation necessary
 - **„Explain”**: Give your answer and explain why you gave this answer.
 - **„Discuss”**: Contrast possible answers and explain the arguments for either of them.
 - ...etc, you should know this from school and other courses.
- Answer the stated question, not more, not less! Time is your enemy and there are no points for brain-dumping a whole topic on the answer sheet.

General info

- The second exam date is set to be September 21st
- The goal is that everybody passes the first exam!
- It might be a little hard to write a really good grade, but it's not hard to pass the exam 😊
- Please remember the team assignment deadline: July 26th, 23:59
- Best case: tagged version in Github Repo (something like "submission")
- Otherwise I will consider the ***last commit before the deadline in the main branch***
- Test everything with a fresh environment!

Contact



For general questions and enquiries on **research, teaching, job openings** and **new projects** refer to our website at www.is3.uni-koeln.de



For specific **enquiries regarding this course** contact us by sending an email to the **IS3 teaching address** at is3-teaching@wiso.uni-koeln.de

To help us process your request efficiently, please use the following subject line format:

[AAA] <Your request subject>